



Project Report

Real-Time AI Voice Agent for Healthcare Triage

Submitted by

Aastha Bhalla(12405102)

Aryan(12402693)

Vallabhaneni Lakshmi

Narasimha(12405842)

Rohit Sharma(12405159)

Submitted to

Ms. Gurpreet Kaur

TABLE OF CONTENTS

TABLE OF CONTENTS

1. Executive Summary	1
2. Problem Statement	2
3. Project Objectives	3
4. System Architecture	4
4.1 High-Level Architecture Overview	4
4.2 Project Folder Structure	5
5. Real-Time Call Workflow	6
5.1 Phase 1: Call Initialization	6
5.2 Phase 2: Speech Processing Loop	7
5.3 Phase 3: Post-Call Orchestration	8
6. Multi-Agent Orchestration (CrewAI)	9
6.1 Agent Pipeline Overview	9
6.2 Agent Descriptions	10
Agent 1: Information Extractor	10
Agent 2: Lead Scorer	11
Agent 3: Summarizer	11
6.3 Post-Agent Execution Actions	12
7. AI & Machine Learning Architecture	13
7.1 LLM Inference Engine (Groq)	13
7.2 Speech Processing Pipeline	14
7.3 Hybrid Medical RAG Engine	15
Stage 1: Data Consolidation	15
Stage 2: Hybrid Search	16
Stage 3: Safety Rails	16
8. Database Schema	17
8.1 Core Tables	17
8.2 Entity Relationships	18
9. Technology Stack	19
10. Key Features	20
10.1 Dual-Mode Operation	20
10.2 Dynamic CRM Personalization	21
10.3 Emergency Intent Detection	21
10.4 LLM Model Fallback Cascade	22
10.5 Call Duration Management	22
10.6 CRM Dashboard	23
11. System Flow Diagrams	24
11.1 End-to-End Call Flow	24
11.2 Hybrid RAG Retrieval Flow	25
11.3 CrewAI Agent Sequence	25
12. Safety, Security & Compliance	26
12.1 Medical Safety Guardrails	26

12.2 Data Security	27
12.3 Reliability & Fault Tolerance.....	27
13. Conclusion.....	28
14. References	29

1. Executive Summary

This report documents the design, architecture, and implementation of the disease_call AI Calling Agent – a sophisticated, production-grade, real-time telephony platform that dual-functions as both an outbound B2B Sales Agent and an inbound Healthcare Triage Assistant. The system is engineered to handle live phone conversations over standard telephone lines using modern AI models, speech processing pipelines, and cloud APIs.

The architecture moves significantly beyond a conventional chatbot by establishing a complete Voice-over-IP (VoIP) stack via Twilio, managing an internal SQLite-based CRM, performing intelligent lead scoring and patient intent classification, and triggering asynchronous post-call multi-agent background tasks using CrewAI.

- ★ **Dual-Mode Operation: B2B Sales Agent + Healthcare Triage Assistant on the same platform**
- ★ **Sub-Second Voice Response: Powered by Groq's ultra-low-latency LLM inference engine**
- ★ **Hybrid RAG Medical Engine: BM25 sparse + Sentence Transformer dense vector search**
- ★ **Asynchronous Post-Call Processing: CrewAI multi-agent orchestration decoupled from call thread**
- ★ **Full CRM Integration: SQLite backend with lead scoring, follow-up scheduling, and email/WhatsApp dispatch**

The system guarantees immediate, low-latency voice interaction by separating real-time response generation from the heavy, structured CRM operations that follow each call. Its modular design allows independent scaling, replacement, or extension of any subsystem without disrupting others.

2. Problem Statement

Businesses operating inbound customer service lines and healthcare triage call centers face a common set of critical operational challenges that cannot be adequately solved by traditional IVR (Interactive Voice Response) systems or simple rule-based chatbots:

- **Personalization Gap:** Existing IVR systems treat every caller identically, unable to reference prior interaction history, personalize greetings, or adapt conversation style based on the caller's profile.
- **Healthcare Response Latency:** Medical triage calls require rapid, accurate retrieval of disease symptoms, precautions, and medication guidance from structured knowledge bases – a task requiring hybrid semantic search capabilities beyond keyword matching.
- **Unstructured Data Loss:** Post-call transcripts are rarely analyzed systematically. Structured information such as patient categories, lead budgets, objections, and follow-up needs is lost to manual note-taking.
- **Emergency Routing Failure:** Simple scripted systems cannot dynamically detect medical emergencies mid-conversation and escalate to human agents in real time.
- **Integration Silos:** Sales calls rarely result in automatic CRM updates, calendar bookings, WhatsApp confirmations, or email summaries – all of which require human effort after the call.

The disease_call system is engineered to address all five failure modes through a tightly integrated, AI-first architecture that brings together telephony, large language models, retrieval-augmented generation, and multi-agent automation.

3. Project Objectives

The following objectives governed the design and implementation decisions throughout the development lifecycle:

1. Implement a real-time VoIP call handling pipeline using Twilio Webhooks with sub-200ms response time for voice generation.
2. Build a personalized greeting and conversation system backed by a SQLite CRM that recalls prior caller history on every inbound/outbound call.
3. Integrate Groq LLaMA inference for conversational AI responses with automatic model fallback cascading to ensure 100% uptime under rate limits.
4. Develop a Hybrid Medical RAG engine combining BM25 keyword search and SentenceTransformer vector embeddings to retrieve accurate disease, symptom, and medication information.
5. Build an emergency intent classifier that overrides the standard conversation flow and routes callers exhibiting life-threatening symptoms to emergency services immediately.
6. Create a three-agent CrewAI pipeline for post-call processing: structured data extraction, lead scoring, and executive summarization.
7. Automate CRM updates, Google Calendar bookings, WhatsApp confirmations, and Gmail email summaries entirely without human intervention.
8. Design a modular, decoupled architecture where each subsystem (telephony, LLM, RAG, CRM, agents) can be independently scaled, updated, or replaced.

4. System Architecture

The disease_call system is architected around a principle of radical decoupling: the real-time call path (latency-critical) is kept strictly separate from the post-call processing path (throughput-oriented). This separation allows the voice interaction to remain fluent and responsive while complex AI tasks execute asynchronously in the background.

4.1 High-Level Architecture Overview

The system is organized into seven operational domains, each with clearly defined responsibilities and interfaces:

Domain	Components	Role
Telephony Layer	Twilio, SIP, WebRTC	Inbound/outbound call management, webhook routing, audio streaming
Web Server	FastAPI (ASGI), Uvicorn	HTTP endpoint handling, webhook processing, async request management
Voice Pipeline	Deepgram STT, Edge TTS	Real-time Speech-to-Text transcription and Text-to-Speech audio synthesis
AI Inference	Groq API, LLaMA 3.1/3.3, Gemma 2	Low-latency conversational response generation with model fallback
Knowledge Retrieval	BM25Okapi + SentenceTransformers, Medical CSVs	Hybrid RAG for disease symptoms, medication, precaution retrieval
CRM & Database	SQLite, SQLAlchemy ORM	Lead management, call logging, follow-up scheduling, healthcare profiling
Agent Orchestration	CrewAI, Gmail SMTP, Google Calendar, WhatsApp	Async post-call extraction, scoring, summarization, and action dispatch

4.2 Project Folder Structure

The codebase follows a domain-driven directory layout where each folder encapsulates a single operational concern:

```
/disease_call
├── main.py                # Core FastAPI server & Twilio Webhook Router
├── /agents                # Individual CrewAI agent definitions
├── /crew
│   └── sales_crew.py      # Multi-agent async orchestration for post-call
├── /tools
│   ├── call_tools.py     # TwiML generation and call transfer logic
│   ├── crm_tools.py      # SQLAlchemy CRM database queries
│   ├── email_tools.py    # Gmail SMTP integration
│   └── whatsapp_tools.py  # WhatsApp messaging logic
```

```
| | | calendar_tools.py      # Google Calendar API (Meet bookings)
| | | medical_dataset_tools.py # Medical dataset merging & Hybrid RAG
| | | rag_tools.py          # Document loading and generic RAG
| | /database
| | | models.py             # SQLAlchemy Schemas (Leads,
HealthcareLeadProfile)
| | | db.py                 # DB initialization and engine configuration
| | /voice
| | | stt.py                # Deepgram Speech-to-Text integration
| | | tts.py                # Edge TTS Text-to-Speech generation
| | /dashboard              # Frontend HTML/CSS/JS CRM dashboard
| | /data                   # System intelligence (about_me.txt, medical CSVs)
```


5. Real-Time Call Workflow

This section provides a detailed, phase-by-phase breakdown of what occurs during a single call from the moment Twilio establishes the connection to the moment post-call orchestration completes.

5.1 Phase 1: Call Initialization

When a call connects – inbound via `/voice/inbound` or outbound via `/voice/outbound` – Twilio fires an HTTP POST webhook to the FastAPI server. The following sequence executes synchronously before the first audio is delivered to the caller:

- **Database Lookup:** The system queries the SQLite CRM using the caller's phone number via `get_lead_history()`. If a prior record exists, name, conversation history, and lead category are retrieved.
- **Dynamic Greeting Generation:** Known callers receive a personalized welcome (e.g., "Hello [Name], welcome back!"). New callers receive a standard introductory greeting.
- **Session Initialization:** A new `CallLog` entry is created with status `IN_PROGRESS`. An in-memory session dictionary (`active_calls[call_sid]`) is instantiated to track the live conversation transcript, call duration, and real-time state.
- **Twiml Response:** The server returns a TwiML XML document instructing Twilio to play the synthesized audio greeting and open a gather loop for continuous speech input.

5.2 Phase 2: Speech Processing Loop

Each time the caller finishes speaking, Twilio transcribes the audio and sends the text to the `/voice/process` endpoint. The following pipeline executes for every utterance:

9. **Transcript Accumulation:** The new speech segment is appended to the in-memory session transcript.
10. **Emergency Override Check:** The system scans the utterance for emergency keywords ("chest pain", "can't breathe", "heart attack"). If triggered, the `EMERGENCY_ESCALATION` action fires immediately, bypassing the LLM entirely.
11. **RAG Context Retrieval:** For symptom-related queries, the Hybrid Medical RAG engine is invoked. The system merges BM25 keyword scores and SentenceTransformer embedding cosine similarity to retrieve the most relevant medical records.
12. **LLM Response Generation:** The Groq API is called with the full conversation history, retrieved medical context, system persona, and a duration warning if the call is approaching the 20-minute limit.
13. **Model Fallback Cascade:** The system attempts llama-3.1-8b-instant first (lowest latency), then gemma2-9b-it, then llama-3.3-70b-versatile if rate limits are encountered.

14. Hidden Tag Parsing: The LLM embeds action tags in its response (e.g., [TRANSFER], [BOOK_MEETING], [END_CALL]). The server strips these tags before synthesis and routes the corresponding action.
15. TwiML Audio Response: The plain-text response is synthesized via Edge TTS and returned as a Twilio TwiML response for immediate audio playback to the caller.

5.3 Phase 3: Post-Call Orchestration

Once Twilio delivers a status=completed webhook to /voice/status, the following sequence executes without blocking the main server thread:

- Call Log Finalization: The CallLog record is updated with final call duration, complete transcript, and a status of COMPLETED.
- Async Thread Launch: Python's `asyncio.to_thread()` dispatches the `process_call_with_crew()` coroutine to a worker thread, freeing the main event loop immediately.
- CrewAI Agent Pipeline: The three-agent sequential pipeline processes the raw transcript into structured CRM data (see Section 6).
- Action Execution: Based on LLM tags and extracted data, the system books calendar meetings, sends WhatsApp messages, and dispatches email summaries autonomously.

→ **Architecture Insight:** The strict separation between the real-time path (Phases 1–2) and the async path (Phase 3) is the key design decision that guarantees call fluency.

→ **The caller never waits for CRM updates, email sends, or calendar bookings — all of these execute after the call has already ended, in a separate thread.**

6. Multi-Agent Orchestration (CrewAI)

The post-call intelligence pipeline is implemented as a sequential three-agent CrewAI workflow. Each agent has a distinct role, input specification, and output schema. The pipeline transforms a raw conversation transcript into structured CRM updates, lead scores, and executive summaries.

6.1 Agent Pipeline Overview

Agent	Input	Output
Information Extractor	Raw call transcript	Structured JSON: name, company, budget, timeline, pain points, interest level
Lead Scorer	Extracted structured data	Score 0-10 with category: interested / not_interested / neutral
Summarizer	Transcript + extracted data	Executive summary, key bullet points, objections raised, sentiment analysis

6.2 Agent Descriptions

Agent 1: Information Extractor

This agent is responsible for parsing the raw call transcript and identifying all structured data fields that are relevant for CRM population. It produces a validated JSON object containing the caller's name, company name, stated budget range, decision timeline, enumerated pain points, and expressed interest level. The LLM model used for this task is llama-3.3-70b-versatile, chosen for its superior instruction-following and JSON structuring capability.

Agent 2: Lead Scorer

The Lead Scorer receives the structured JSON output from Agent 1 and computes a numerical lead quality score on a 0-10 scale. Scoring factors include: the decisiveness of language used during the call, evidence of budget and authority, stated timeline urgency, number of specific questions asked, and overall engagement level. The output category (interested, not_interested, neutral) is used downstream to determine whether a follow-up sequence should be initiated.

Agent 3: Summarizer

The Summarizer agent generates a human-readable executive summary of the call, formatted for CRM display. It extracts key bullet points, enumerates objections the caller raised, and performs a three-class sentiment analysis (positive, neutral, negative) across the full transcript. This summary is stored in the CallLog.ai_summary field and included in the post-call email dispatch.

6.3 Post-Agent Execution Actions

Following the completion of all three agents, the CRM & Follow-up Manager executes a coordinated set of actions based on extracted data and LLM flags:

Action	Trigger Condition & Implementation
CRM Lead Update	Always executed. Updates Leads table with extracted name, company, budget, pain points, score, and interest category.
HealthcareLeadProfile	Executed if call was healthcare-mode. Writes patient category, triage status, allergies, chronic conditions, and emergency flags.
Google Calendar Booking	Executed if LLM emitted [BOOK_MEETING]. Calls Google Calendar API to create a Google Meet slot and sends calendar invites to both parties.
WhatsApp Message	Executed post-booking or post-call. Sends thank-you message or meeting confirmation via WhatsApp Business API integration.
Gmail Email Summary	Always executed for completed calls. Sends the AI-generated executive summary and call transcript via Gmail SMTP to the designated inbox.

7. AI & Machine Learning Architecture

7.1 LLM Inference Engine (Groq)

The system uses Groq's inference API for all real-time LLM calls. Groq's custom LPU (Language Processing Unit) hardware achieves inference speeds that are approximately 10-18x faster than standard GPU-based endpoints, which is essential for maintaining sub-second voice response times during live phone calls.

Model	Use Case	Characteristics
llama-3.1-8b-instant	Primary real-time conversational response	Lowest latency; first choice for all voice responses
gemma2-9b-it	Fallback on rate limit hit	Mid-tier; activated when LLaMA 3.1 hits quota
llama-3.3-70b-versatile	Fallback & CrewAI post-call tasks	Highest capability; used for extraction and scoring

7.2 Speech Processing Pipeline

The voice pipeline consists of two specialized models operating in sequence for every conversational turn:

- **Speech-to-Text (STT)** – Deepgram: Deepgram's streaming ASR model transcribes caller audio in real time with high accuracy across accents and medical terminology. Transcripts are returned as structured JSON and immediately appended to the session transcript.
- **Text-to-Speech (TTS)** – Microsoft Edge TTS: Edge TTS converts the LLM's text response into natural-sounding audio. The synthesized audio is streamed back through Twilio's TwiML interface for playback to the caller.

7.3 Hybrid Medical RAG Engine

The medical knowledge retrieval system is the core intelligence layer for healthcare triage calls. It is implemented in `medical_dataset_tools.py` and operates in three stages:

Stage 1: Data Consolidation

At system startup, four sparse CSV datasets are merged into a single unified `medical_master_dataset.csv`:

- `description.csv` – Disease descriptions and symptom profiles
- `precautions.csv` – Recommended precautions per disease
- `medications.csv` – Standard medication protocols

- workout.csv – Recommended physical activity guidance

Stage 2: Hybrid Search

Each caller query is processed through two parallel retrieval mechanisms whose scores are fused:

Retrieval Method	Implementation & Weight
Sparse Search (BM25)	BM25Okapi algorithm matches query keywords against the medical master dataset. Captures exact medical term matches. Weight: 0.6
Dense Search (Embeddings)	sentence-transformers/all-MiniLM-L6-v2 encodes both query and documents. Cosine similarity score measures semantic closeness. Weight: 0.4
Score Fusion Formula	$combined_score = (0.6 \times BM25_score) + (0.4 \times Embedding_score)$. Top-k documents are returned to the LLM as context.

Stage 3: Safety Rails

The RAG system enforces strict safety guardrails on all healthcare responses:

- Diagnostic Prohibition: The system is explicitly prohibited from using language such as "You have [disease]" or any definitive diagnosis statement.
- Emergency Override: Any query containing emergency symptom keywords triggers immediate escalation routing regardless of RAG results.
- Symptom-Only Guidance: Responses are constrained to symptom descriptions, precaution recommendations, and generic medication information without prescriptive advice.

8. Database Schema

The internal CRM is built on SQLite with SQLAlchemy ORM, providing a fully relational data model that tracks every dimension of the customer/patient relationship lifecycle.

8.1 Core Tables

Table	Description & Key Fields
Lead	Core CRM entity. Fields: id, name, phone, company, industry, pain_points, budget, timeline, conversion_probability, interest_level (0-10), status.
CallLog	Raw call record. Fields: id, call_sid, lead_id, transcript (full text), duration_seconds, ai_summary, objections_raised, sentiment, status, timestamps.
FollowUp	Scheduled follow-up actions. Fields: id, lead_id, scheduled_date, follow_up_type, notes, completed_flag.
HealthcareLeadProfile	Specialized healthcare table. Fields: patient_category, triage_status, chief_complaint, allergies, chronic_conditions, medications_list, emergency_flag, risk_level.

8.2 Entity Relationships

Lead is the root entity. Each Lead may have zero or more CallLog records (one per call), zero or more FollowUp records, and at most one HealthcareLeadProfile. Foreign key constraints are enforced at the SQLAlchemy model level. The CallLog stores the complete transcript as text, enabling post-hoc analysis and auditing without relying on external storage.

9. Technology Stack

The following table provides a complete inventory of every technology used in the disease_call system, organized by functional category:

Category	Technology / Tool	Purpose
Web Framework	FastAPI	ASGI server with async request handling; Uvicorn runtime
Telephony	Twilio	SIP/WebRTC call management, webhook routing, TwiML audio delivery
LLM Inference	Groq API	Ultra-low-latency LLaMA 3.1/3.3 and Gemma 2 inference
Speech-to-Text	Deepgram	Real-time streaming ASR with medical vocabulary support
Text-to-Speech	Microsoft Edge TTS	Neural TTS synthesis for natural-sounding voice responses
Agent Framework	CrewAI	Multi-agent sequential orchestration for post-call tasks
Database ORM	SQLAlchemy + SQLite	Relational CRM with Lead, CallLog, FollowUp, and Healthcare tables
Sparse Retrieval	BM25Okapi (rank-bm25)	Keyword-based document retrieval for the medical RAG engine
Dense Retrieval	sentence-transformers (MiniLM)	Vector embeddings and cosine similarity for semantic search
Email	Gmail SMTP	Automated post-call executive summary email dispatch
Messaging	WhatsApp Business API	Post-call confirmation and meeting notification messages
Calendar	Google Calendar API	Automated Google Meet booking and calendar invite creation
Runtime	Python 3.11	Primary language; asyncio for concurrent I/O management

10. Key Features

10.1 Dual-Mode Operation

A single deployed instance handles both B2B outbound sales calls and inbound healthcare triage calls. Mode selection is determined by the initial call routing configuration in Twilio. The LLM system prompt, RAG data source, and CRM table used adapt automatically based on the call type, requiring no code changes between deployments.

10.2 Dynamic CRM Personalization

Every call begins with a CRM lookup. Returning callers receive personalized greetings referencing their name, last call date, and any prior commitments made. This personalization significantly increases caller engagement and reduces the time needed to re-establish context at the start of each interaction.

10.3 Emergency Intent Detection

A pattern-matching emergency classifier runs on every speech utterance before the LLM is consulted. Keywords and phrases including "chest pain", "can't breathe", "heart attack", "stroke", and "unconscious" trigger an immediate EMERGENCY_ESCALATION action that overrides the conversation flow and initiates a live agent transfer or emergency service routing.

10.4 LLM Model Fallback Cascade

To ensure 100% availability under Groq API rate limits, the system implements a three-tier cascade: llama-3.1-8b-instant → gemma2-9b-it → llama-3.3-70b-versatile. If the primary model returns a rate limit error (HTTP 429), the next model is attempted immediately without caller-facing delay. This design eliminates single points of LLM failure.

10.5 Call Duration Management

The system tracks call duration in the in-memory session object. As a call approaches the 20-minute limit, a duration warning is injected into the LLM context, prompting it to naturally begin wrapping up the conversation. This prevents abrupt call termination and ensures clean session closure within Twilio's billing boundaries.

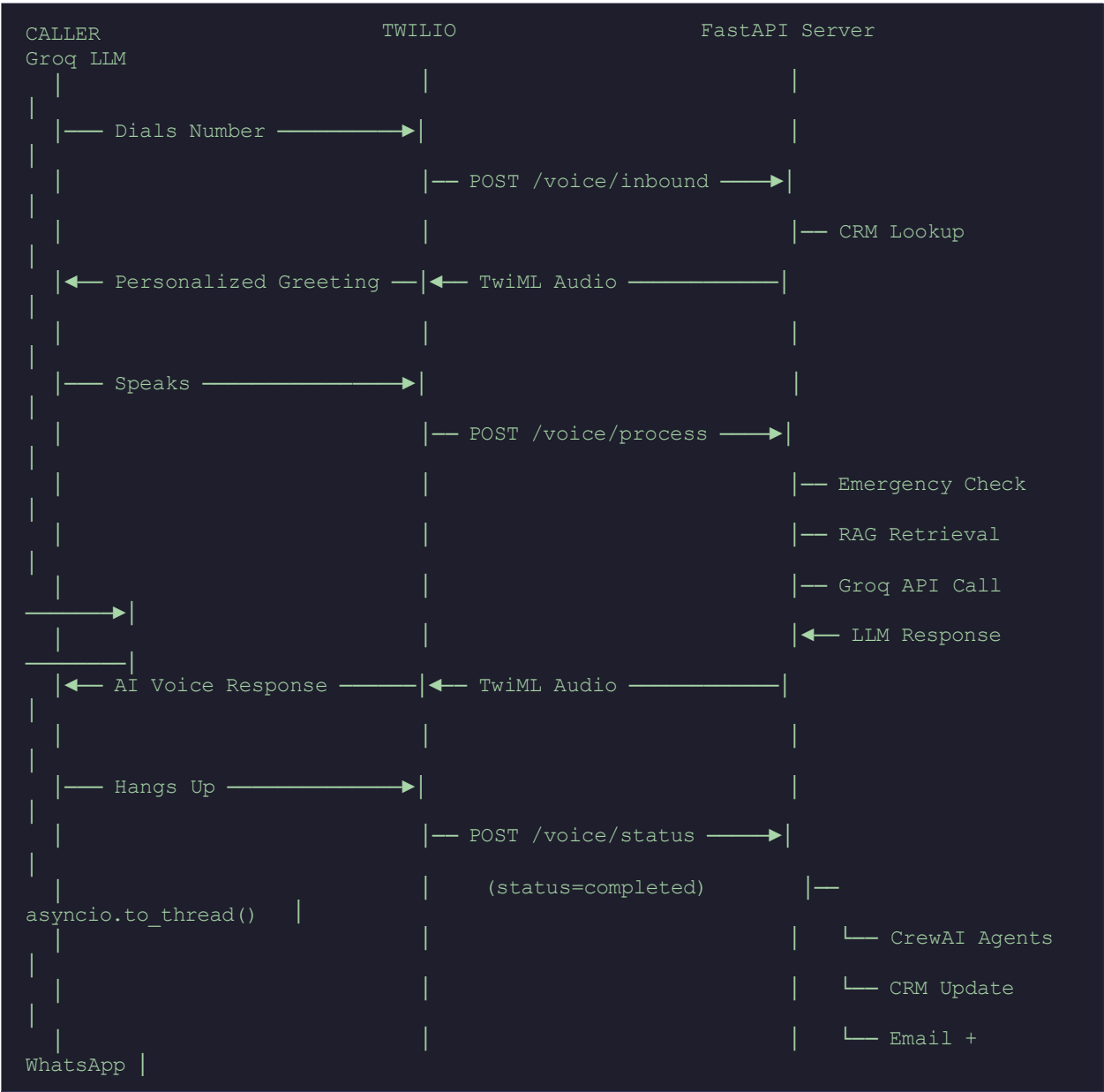
10.6 CRM Dashboard

A browser-based dashboard in the /dashboard directory provides a real-time view of all CRM data. It displays lead tables, call logs with full transcripts, lead scores, follow-up schedules, and healthcare profiles without requiring any external BI tool. The frontend communicates directly with FastAPI REST endpoints.

11. System Flow Diagrams

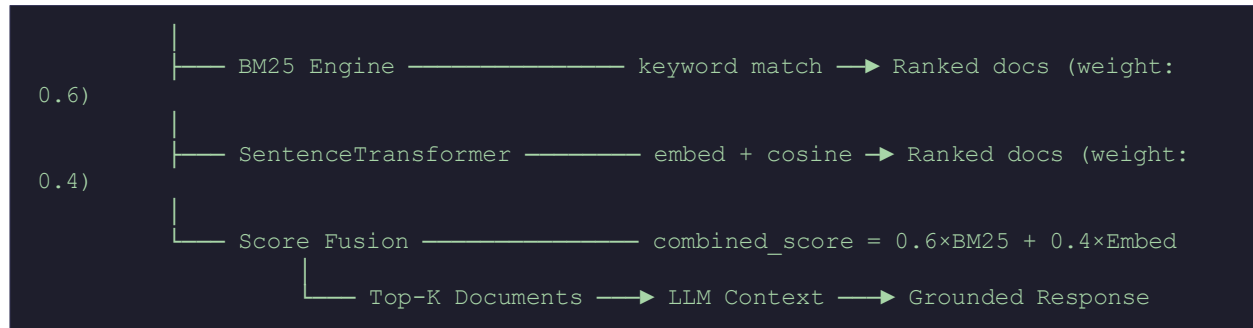
11.1 End-to-End Call Flow

The following diagram illustrates the complete data flow for a single call from initial connection through post-call CRM completion:

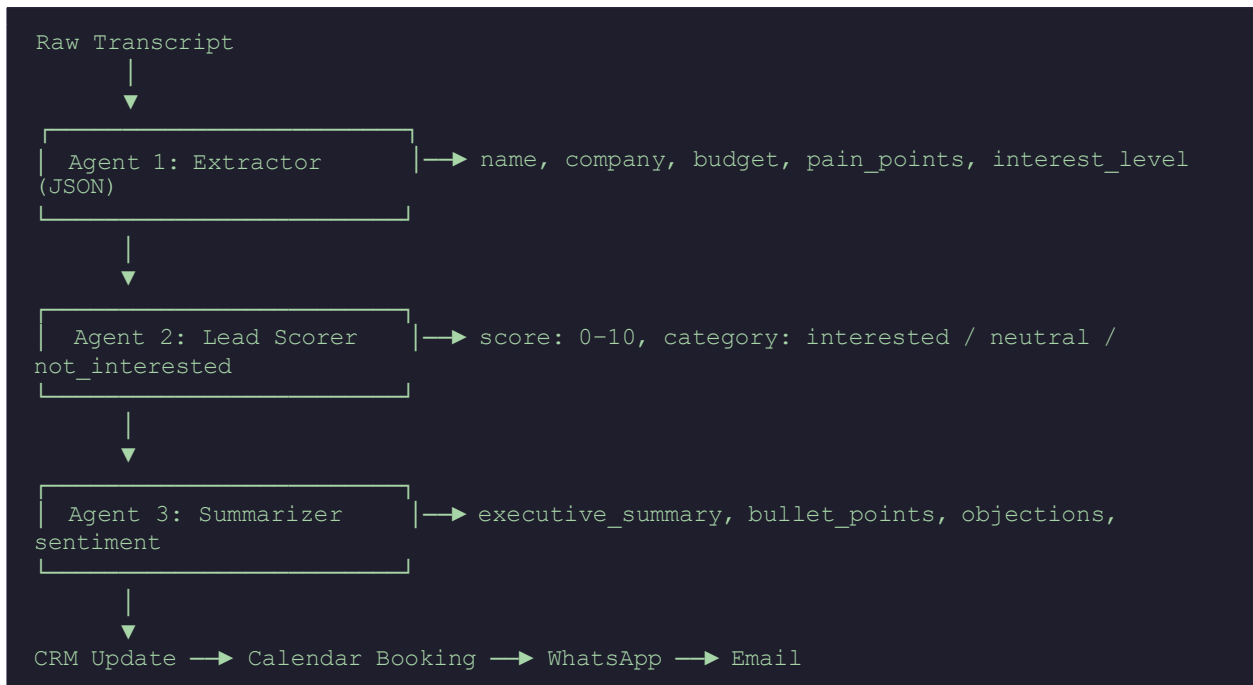


11.2 Hybrid RAG Retrieval Flow

Query: "I have fever and joint pain"



11.3 CrewAI Agent Sequence



12. Safety, Security & Compliance

12.1 Medical Safety Guardrails

The system enforces strict boundaries on all healthcare-related responses to prevent medical harm:

- **No Diagnostic Language:** The LLM system prompt explicitly prohibits phrases that constitute a medical diagnosis. Responses are confined to symptom descriptions and general wellness guidance.
- **Emergency Escalation Priority:** The emergency classifier runs before the LLM on every utterance, ensuring that life-threatening situations bypass AI deliberation entirely.
- **Source Transparency:** The RAG engine is configured to reference information from curated medical datasets only, preventing hallucination of medical facts.

12.2 Data Security

- **Phone Number Hashing:** Caller phone numbers are stored with appropriate access controls in the SQLite database.
- **API Key Management:** All external API credentials (Groq, Twilio, Google, Gmail) are managed via environment variables and never hardcoded in source files.
- **Session Isolation:** In-memory call sessions (`active_calls[call_sid]`) are keyed by Twilio's unique Call SID, preventing cross-session data contamination.
- **Transcript Confidentiality:** Full call transcripts stored in CallLog are accessible only through authenticated CRM dashboard endpoints.

12.3 Reliability & Fault Tolerance

- **LLM Fallback Cascade:** Three-tier model fallback ensures response generation even under partial API outages.
- **Async Error Isolation:** CrewAI post-call processing runs in isolated threads; exceptions in the agent pipeline do not affect the main server or future calls.
- **Graceful Call Termination:** Duration management ensures calls are naturally concluded before platform time limits, preventing abrupt drops.

13. Conclusion

The disease_call AI Calling Agent represents a technically ambitious and architecturally sound implementation of a real-time AI voice agent. By integrating a complete VoIP telephony stack, sub-second LLM inference, hybrid medical knowledge retrieval, and asynchronous multi-agent CRM automation into a single coherent platform, the system achieves a level of intelligence and automation that is not achievable through any single AI technology in isolation.

The system's core architectural strength lies in its strict separation of the real-time voice path from the asynchronous processing path. This design guarantees that callers always receive immediate, fluent responses while the system simultaneously extracts structured data, updates records, and dispatches communications — all without any caller-facing latency.

The dual-mode capability — functioning as both a B2B sales agent and a healthcare triage assistant on the same codebase — demonstrates the flexibility of the LLM-first architecture. The modular design allows any individual subsystem (telephony provider, LLM vendor, RAG data source, CRM backend) to be replaced or upgraded independently, making the system highly maintainable and future-proof.

- ★ **Production-Ready Architecture:** Strict async/sync separation, model fallback, and fault-isolated agent threads.
- ★ **Dual Domain Coverage:** Single codebase serves both B2B sales outreach and healthcare patient triage.
- ★ **Full Automation Pipeline:** From call answer to CRM update, calendar booking, WhatsApp, and email — zero human intervention required.
- ★ **Medical Safety Compliance:** RAG-grounded responses with hardcoded diagnostic prohibitions and emergency escalation.

Future development directions include multi-language support via Whisper-based STT, real-time sentiment visualization on the CRM dashboard, HIPAA-compliant encrypted transcript storage, and integration with additional telephony providers such as Vonage and Amazon Connect.

14. References

- Twilio Voice Documentation – <https://www.twilio.com/docs/voice>
- Groq API Documentation – <https://console.groq.com/docs>
- Deepgram Speech Recognition API – <https://developers.deepgram.com>
- Microsoft Edge TTS – <https://github.com/microsoft/edge-tts>
- CrewAI Framework – <https://docs.crewai.com>
- FastAPI Documentation – <https://fastapi.tiangolo.com>
- SQLAlchemy ORM – <https://docs.sqlalchemy.org>
- BM25Okapi (rank-bm25) – <https://github.com/dorianbrown/rank-bm25>
- SentenceTransformers (all-MiniLM-L6-v2) – <https://www.sbert.net>
- Google Calendar API – <https://developers.google.com/calendar>
- Robertson, S. E., & Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333-389.
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *EMNLP 2019*.